

Adding Generics

- Ugly to use Object with Measurable
- Make parameter generic:

```
public interface Measurer<T>
{
    public double getMeasure(T obj);
}
```

- Then make utility method generic:

```
public static <T> double average(T[] objects, Measurer<T> meas)
{
    double sum = 0;
    for (T obj : objects) ...
    ...
}
```

- Actually, it should be `Measurer<? super T>`, but we'll ignore that for now.

Comparator.comparing

- Look again at

```
Arrays.sort(words, (a, b) -> a.length() - b.length());
```

- How would we sort bank accounts by balance?

```
Arrays.sort(accounts, (a, b) -> v.getBalance() - w.getBalance());
```

- Ok, not quite. Need to use `Double.compare`:

```
(a, b) -> Double.compare(v.getBalance(), w.getBalance())
```

- That's a general principle—the comparator applies a measurement and compares the outputs.
- The static method `Comparator.comparing` yields such a comparator:

```
Arrays.sort(words, Comparator.comparing(w -> w.length()));
```

- Aside: As of Java 8, interfaces can have static methods. They work just like static methods in classes.

What Does Comparator.comparing Do?

- It receives a “function”
 - (really, an object of some class that implements a functional interface)
- It returns a “function”
 - (really, an object...)
- It's a “higher-order function”
- Like this:

```
public static <T> Comparator<T> comparing(Measurer<T> meas)
{
    return (a, b) -> Double.compare(
        meas.getMeasure(a), meas.getMeasure(b));
}
```

- Actually, in the Java API, it doesn't use a `Measurer` but a `Function`

```
public interface Function<T, R>
{
    R apply(T arg)
}
```

- Specifically,

```
static <T,U extends Comparable<? super U>>
    Comparator<T> comparing(Function<? super T,? extends U> measure)
```

Higher-Order Functions

- A “function” that consumes and/or produces “functions”
- Example: `filter` consumes function

```
public static <T> List<T> filter(List<T> values, Predicate<T> p)
{
    List<T> result = new ArrayList<>();
    for (T value : values)
    {
        if (p.test(value)) { result.add(value); }
    }
    return result;
}
```

- Here, `Predicate` is a standard functional interface:

```
public interface Predicate<T>
{
    boolean test(T arg);
}
```

- Typical use:

```
List<String> longWords = filter(wordList, w -> w.length() > 10);
```

Constructor Expressions

- Like method expression, with special method name `new`
- For example,

```
BankAccount::new
```

- is equivalent to a lambda expression that invokes the `BankAccount` constructor.

- Which constructor?

- Depends on context—could be

```
() -> new BankAccount()
```

- or

```
b -> new BankAccount(b)
```

- Constructor expressions can construct arrays:

```
String[]::new
```

- Same as

```
(n: int) -> new String[n]
```

- Used to overcome limitation of Java generics—can't construct array of generic type.

Method Expressions

- Common to have lambda expressions that just invoke a method
- Use *method expression*: `ClassName::methodName`

```
String::toUpperCase
```

- Parameters are added “at the right places.”

```
(String w) -> w.toUpperCase()
```

- If method has a parameter, the lambda expression gets two parameters:

```
String::compareTo
```

is the same as

```
(String s, String t) -> s.compareTo(t)
```

- Also works with static methods:

```
Double::compare
```

is the same as

```
(double x, double y) -> Double.compare(x, y)
```

- Can have an object to the left of `::` symbol:

```
System.out::println
```

is the same as

```
x -> System.out.println(x)
```

Method Expressions and Comparators

- `Comparator.comparing` makes a comparator from an extractor function:

```
Comparator<String> comp = Comparator.comparing(t -> t.length())
```

- Same as

```
Comparator<String> comp = (v, w) -> v.length() - w.length();
```

- Note that the extractor function makes a single method call
- Write as method reference:

```
Comparator.comparing(String::length)
```

- Reads nicely: the comparator that compares strings by their length.
- Can add a secondary comparison with `thenComparing`:

```
Collections.sort(countries,
    Comparator.comparing(Country::getContinent)
        .thenComparing(Country::getName));
```

- Countries are compared first by continent.
- If the continents are the same, they are compared by name.

- Easy to read, easy to write
- Thanks to lambda expressions, method expressions, higher order functions

Some Facts About Streams

- Streams are similar to collections, but...
- They don't store their own data
 - The data comes from elsewhere
 - From a collection, a file, a database, data from the internet, ...
- Streams were designed to work well with lambda expressions

```
stream.filter(w -> w.length() > 10)
```

- Streams are immutable
 - Methods such as `filter` produce new streams
- Stream processing is *lazy*—see the next slide

Lazy Processing

- Instead of counting the words, let's see some (but not all)
- Here is how to get the first five:

```
Stream<String> fiveLongWords = words
    .filter(w -> w.length() > 10)
    .limit(5);
```

- Dumb approach: first generate all long words, and throw most of them away.
- Fortunately, stream processing is not dumb but *lazy*
 - Works “backwards” and only computes what is necessary.
- `limit(5)` needs an element...
- ... and `filter(...)` examines elements until it finds one.
- That repeats another four times.
- And then...nothing.
- The other stream elements never get examined.

Producing Streams

- In order to process streams, you first need to have one
- Simplest way: the `of` method

```
Stream<String> words = Stream.of("Mary", "had", "a", "little", "lamb");
Stream<Integer> digits = Stream.of(3, 1, 4, 1, 5, 9);
```

- Also works for arrays:

```
Integer[] digitArray = { 3, 1, 4, 1, 5, 9 };
Stream<Integer> digitStream = Stream.of(digitArray);
```

- This is a stream of `Integer` objects
- You'll see later how to make a stream of `int`

- Any collection can be turned into a stream:

```
List<String> wordList = new ArrayList<>();
// Populate wordList
Stream<String> words = wordList.stream();
```

Producing Streams 2

- Several utility methods yield streams:

```
String filename = . . . ;
try (Stream<String> lineStream = Files.lines(Paths.get(filename)))
{
    ...
} // File is closed here
```

- You can make infinite streams:

```
Stream<Integer> integers = Stream.iterate(0, n -> n + 1);
```

- You can turn any stream into a *parallel stream*

- `Stream<String> parStream = lineStream.parallel()`
- Operations such as `filter` and `count` run in parallel, each processor working on chunks of the data

Collecting Results 1

- When you are done transforming a stream (e.g. with `filter`), want to harvest results
- Some methods (e.g. `count`, `sum`) yield a single value
- Other methods yield a collection
- Here is how to collect into an array:

```
String[] result = stream.toArray(String[]::new);
```

- Strange-looking expression `String[]::new` is a constructor reference (to the array constructor)
- Replace `String` with another class if the stream doesn't contain strings

Collecting Results 2

- To collect into a `List` or `Set`, use `collect`:

```
List<String> result = stream.collect(Collectors.toList());
Set<String> result = stream.collect(Collectors.toSet());
```

- The argument to `collect` is a `Collector` object
- We'll always use one of the static method of `Collectors` to get one

- A stream of string can be collected into a single string:

```
String result = words.collect(Collectors.joining(", "));
// Stream elements separated with commas
```

Stream Tranformations: map

- Life cycle of a stream:

- Create stream
- Transform stream (possibly multiple times)
- Collect results

- `map` transforms stream by applying function to each element

- Turn all words into lowercase:

```
Stream<String> words = Stream.of("A", "Tale", "of", "Two", "Cities");
Stream<String> lowerCaseWords = words.map(w -> w.toLowerCase());
// "a", "tale", "of", "two", "cities"
```

- Remove vowels from all words:

```
Stream<String> consonantsOnly = lowerCaseWords.map(
    w -> w.replaceAll("[aeiou]", ""));
// "", "tl", "f", "tw", "cts"
```

- Get the length of each element:

```
Stream<Integer> consonantCount = consonantsOnly.map(w -> w.length()); // 0, 2, 1, 2, 3
```

More Stream Transformations

- Applying `map` yields a stream with the same number of elements
- `filter` only retains matching elements:

```
Stream<String> aWords = words.filter(w -> w.substring(0, 1).equals("a"));
// Only the words starting with "a"
```

- `limit` takes the first `n`:

```
Stream<String> first100aWords = aWords.limit(100);
```

- `skip` takes all but the first `n`:

```
Stream<String> allButFirst100aWords = aWords.skip(100);
```

- `distinct` yields a stream with duplicates removed:

```
Stream<String> words = Stream.of(
    "how much wood could a wood chuck chuck".split(" "));
Stream<String> distinctWords = words.distinct();
// "how", "much", "wood", "could", "a", "chuck"
```

- `sorted` yields a new stream in which the elements are sorted:

```
Stream<String> sortedWords = distinctWords.sorted();
// "a", "chuck", "could", "how", "much", "wood"
```

- Element type must be `Comparable`
- Or supply a comparator: `distinctWords.sorted((s, t) -> s.length() - t.length())`

Creating Primitive-Type Streams

- Can create from individual numbers, or an array

```
IntStream stream = IntStream.of(3, 1, 4, 1, 5, 9);
int[] values = . . . ;
stream = IntStream.of(values);
```

- `range` yields a contiguous range of integers

```
IntStream stream = IntStream.range(a, b);
// Stream contains a, a + 1, a + 2, ..., b - 1
```

- Random generator yields infinite stream of random numbers

```
Random generator = new Random();
IntStream dieTosses = generator.ints(1, 7);
```

- String yields stream of Unicode code points

```
IntStream codePoints = str.codePoints();
```

- Aside: This is the best way of getting the code points of a string in the Java API!
- Much better than `charAt` which only yields 16-bit code units of the variable-length UTF-16 encoding

Common Error: Don't Use a Terminated Stream

- Once you apply a terminal operation, a stream is "used up"
- Can no longer apply any stream operations
- Easy mistake if you have a reference to an intermediate stream:

```
Stream<String> stream = list.stream();
List<String> result1 = stream.limit(50).collect(Collectors.toList());
// Save the first fifty
stream = stream.skip(50);
// Error—the stream can no longer be used
```

- To avoid the error, use "pipeline notation"

```
result = create(...)
    .transform1(...)
    .transform2(...)
    .terminalOperation(...)
```

- If you want to do what the example tried to do, you need two streams.

```
List<String> result1 = list.stream()
    .limit(50)
    .collect(Collectors.toList()); // This stream can no longer be used
List<String> result2 = list.stream() // Create another stream
    .skip(50)
    .limit(50)
    .collect(Collectors.toList());
```

Common Error: Don't Use a Terminated Stream

- Once you apply a terminal operation, a stream is "used up"
- Can no longer apply any stream operations
- Easy mistake if you have a reference to an intermediate stream:

```
Stream<String> stream = list.stream();
List<String> result1 = stream.limit(50).collect(Collectors.toList());
// Save the first fifty
stream = stream.skip(50);
// Error—the stream can no longer be used
```

- To avoid the error, use "pipeline notation"

```
result = create(...)
    .transform1(...)
    .transform2(...)
    .terminalOperation(...)
```

- If you want to do what the example tried to do, you need two streams.

```
List<String> result1 = list.stream()
    .limit(50)
    .collect(Collectors.toList()); // This stream can no longer be used
List<String> result2 = list.stream() // Create another stream
    .skip(50)
    .limit(50)
    .collect(Collectors.toList());
```

Working with Optional

- Work with it, not against it. (That is, don't treat it like a potentially null reference.)
- `orElse` extracts the value or an alternative if there is none:

```
int length = optResult.orElse("").length();
```

- `ifPresent` passes on the value to a function; does nothing if there is none

```
optResult.ifPresent(v -> results.add(v));
```

- If neither of these works, use `isPresent` to test if there is a value and `get` to get it.

```
if (optResult.isPresent())
{
    System.out.println(optResult.get());
}
else
{
    System.out.println("No element");
}
```

Other Terminal Operations

- `findAny` is like `findFirst`, but is faster on parallel streams.

```
result = words
    .parallel()
    .filter(w -> w.length() > 10)
    .filter(w -> w.endsWith("y"))
    .findAny()
    .orElse("");
```

- `max/min` require a comparator and return an `Optional` (since the input may be empty):

```
Optional<String> result = words.max((v, w) -> v.length() - w.length());
```

- `allMatch/anyMatch/noneMatch` check a predicate:

```
boolean result = words.allMatch(w -> w.contains("e"));
// result is true if all words contain the letter e
```

Primitive-Type Streams

Inefficient to have streams of number wrappers such as `Stream<Integer>`

For example, `numbers.map(x -> x * x)` requires unboxing and boxing for each element

`IntStream`, `DoubleStream`, `DoubleStream` work with `int`, `long`, `double` values without boxing

No stream classes for `byte`, `short`, `long`, `char`, `float`

Mapping Primitive-Type Streams

- `IntStream.map` with an `int -> int` function yields another `IntStream`

```
IntStream stream = IntStream.range(0, 20)
    .map(n -> Math.min(n, 10));
// A stream with twenty elements 0, 1, 2, ..., 9, 10, 10, ..., 10
```

- When the function yields objects, use `mapToObj`:

```
String river = "Mississippi";
int n = river.length();
Stream<String> prefixes = IntStream.range(0, n)
    .mapToObj(i -> river.substring(0, i));
// "", "M", "Mi", "Mis", "Miss", "Missi", ...
```

- Also have `mapToDouble`, `mapToLong` if the function yields `double`, `long` values
- Think of `IntStream.range(a, b).mapXXX` as an equivalent of a `for` loop that yields a value in each iteration.
- Use `mapToInt`/`mapToLong`/`mapToDouble` with streams of objects when the `map` function yields primitive type values
- Use `boxed` to turn into a stream of objects

Processing Primitive-Type Streams

- Stream methods for primitive-type streams have modified parameters/return types
- For example, `IntStream.toArray` returns an `int[]` and doesn't require a constructor
- Four additional methods `sum`, `average`, `max`, `min` (without comparators)
- Last three return `OptionalInt`/`OptionalLong`/`OptionalDouble`

```
double average = words
    .mapToInt(w -> w.length())
    .average()
    .orElse(0);
```